



Java

Clase 06 | Encapsulamiento y colaboración entre clases

Índice

Clase 06

Clase 6 | Encapsulamiento y colaboración entre clases

- Encapsulamiento y visibilidad (public, private, protected).
- Métodos de acceso (getters y setters).
- Colaboración entre clases.
- Objetos dentro de objetos.

Variables de clase (static).

Clase 07

Clase 7 | Herencia y polimorfismo

- Concepto de herencia en Java.
- Creación de subclases y superclases.
- Uso de super y this.
- Polimorfismo y sobreescritura de métodos.
- Clases abstractas e interfaces.
- Ejemplos prácticos aplicados al proyecto.

Objetivos de la Clase



Encapsulamiento

Entender visibilidades public, private y protected.



Colaboración

Explorar la interacción entre clases y objetos.



Métodos de Acceso

Aprender a usar getters y setters.



Variables de Clase

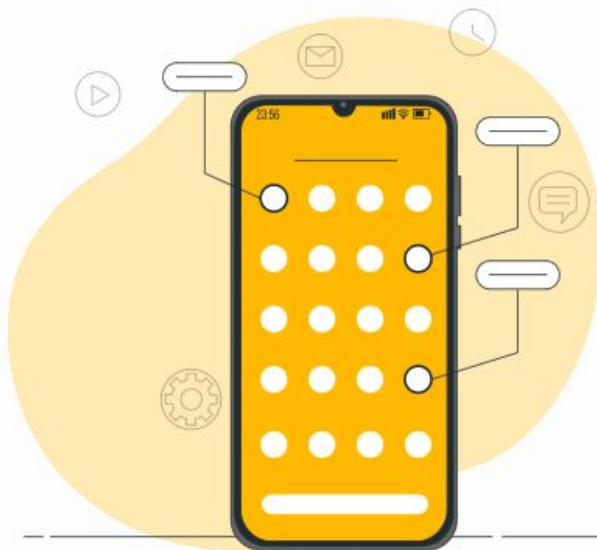
Comprender el uso de variables estáticas.

Encapsulamiento

Introducción a Encapsulamiento

El encapsulamiento es un principio fundamental en la programación orientada a objetos que protege los datos de una clase y controla el acceso a ellos. En Java, este control se logra a través de modificadores de acceso como **public**, **private** y **protected**.

La visibilidad **public** permite que los datos sean accesibles desde cualquier parte del programa. La visibilidad **private** limita el acceso solo a la clase misma. La visibilidad **protected** permite el acceso a la clase y a sus subclases.



Tipos de Visibilidad

Public

Permite acceso total desde cualquier clase o paquete del programa. Ofrece máxima visibilidad pero menor encapsulamiento.

Private

Restringe el acceso exclusivamente a los métodos de la propia clase. Proporciona el máximo nivel de encapsulamiento y seguridad.

Protected

Permite acceso desde la misma clase, sus clases heredadas y otras clases del mismo paquete. Ofrece un nivel intermedio de protección.



Métodos de Acceso: Getters y Setters

Los getters y setters son métodos especiales que nos permiten acceder y modificar los atributos privados de una clase de manera controlada y segura.

Getter

Método que permite obtener el valor de un atributo privado de forma controlada. Por ejemplo: `getNombre()` retorna el nombre del objeto. Estos métodos siguen la convención "get" + nombreAtributo y garantizan el acceso seguro a los datos encapsulados.

Setter

Método que permite modificar el valor de un atributo privado de manera segura. Por ejemplo: `setEdad(int edad)` verifica que la edad sea válida antes de asignarla. Estos métodos siguen la convención "set" + nombreAtributo y son fundamentales para mantener la integridad de los datos.

Ejemplo: Getters y Setters en Producto

En este ejemplo, vemos cómo implementar getters y setters para controlar el acceso al precio de un producto. El setter incluye una validación que asegura que el precio nunca pueda ser negativo, demostrando cómo el encapsulamiento nos permite proteger la integridad de los datos.

```
public class Producto {  
    private double precio;  
  
    public double getPrecio() {  
        return precio;  
    }  
  
    public void setPrecio(double precio) {  
        if (precio >= 0) {  
            this.precio = precio;  
        }  
    }  
}
```


¿Y por qué necesitamos proteger nuestros datos?

1

Código Vulnerable

Cuando los atributos son públicos, cualquier parte del programa puede modificarlos directamente. Por ejemplo: si precio es público, alguien podría escribir `producto.precio = -100` y crear un precio negativo inválido.

2

Solución

Hacemos los atributos privados (`private`) y creamos métodos específicos para accederlos. Por ejemplo: `setPrecio()` puede verificar que el precio sea positivo antes de asignarlo.

3

Beneficios

Nuestros datos están protegidos contra errores. Solo podemos modificarlos usando métodos que garantizan que los valores sean correctos. El código es más seguro y fácil de mantener.

Control de Atributos

Sin Encapsulamiento (Incorrecto)

Aquí el acceso es directo a la variable, sin protección.

```
public class Producto {  
    public double precio;  
}  
  
// Uso incorrecto  
Producto p = new Producto();  
p.precio = -100; // Permite valores inválidos
```

✗ Acceso directo

El acceso directo a los atributos permite asignar valores inválidos y rompe el encapsulamiento.

Con Encapsulamiento (Correcto)

```
public class Producto {  
    private double precio;  
  
    public double getPrecio() { return precio; }  
  
    public void setPrecio(double precio) {  
        if (precio >= 0) {  
            this.precio = precio;  
        }  
    }  
}  
  
// Uso correcto  
Producto p = new Producto();  
p.setPrecio(100); // Validación garantizada
```

✓ Atributo protegido

Aquí accedemos por getter & setter y además chequeamos que el valor sea válido

Colaboracion entre clases

Colaboración entre Clases

En el desarrollo de software real, nos enfrentamos a sistemas complejos que pueden contener cientos o incluso miles de clases interactuando entre sí. La colaboración entre clases se vuelve fundamental para gestionar esta complejidad.

La clave está en diseñar estas relaciones de manera efectiva, estableciendo conexiones claras y lógicas entre las clases. Por ejemplo, en un sistema de ventas, la clase Pedido no solo se relaciona con Producto, sino también con Cliente, Inventario, Facturación y muchas más, formando una red compleja de colaboración que refleja la realidad del negocio.



Problema: Sistema de Gestión de Pedidos

-  Una tienda en línea necesita implementar un sistema para gestionar pedidos que contengan múltiples productos. El sistema debe permitir:
- Crear nuevos pedidos para los clientes
 - Agregar varios productos diferentes a un mismo pedido
 - Calcular el total del pedido sumando el precio de todos los productos incluidos

¿Cómo podemos implementar esta funcionalidad utilizando la colaboración entre las clases Pedido y Producto?

```
// La clase Pedido contiene una lista de
// objetos Producto
public class Pedido {
    private ArrayList productos;

    public void agregarProducto(Producto p)
    {
        productos.add(p);
    }

    public double calcularTotal() {
        // Lógica para sumar precios
    }
}
```

Variables Estáticas



Variables estáticas

Una variable estática es una variable que pertenece a la clase en sí, en lugar de a un objeto específico. Esto significa que existe solo una copia de la variable estática, compartida por todos los objetos de la clase.

Se pueden usar para almacenar valores compartidos por todos los objetos de una clase, como un contador de instancias, valores predeterminados, o incluso datos de configuración.

Ejemplo: Contador de Productos

En este ejemplo, utilizamos una variable estática **contadorProductos** para llevar un registro del número total de productos creados. Cada vez que se crea un nuevo objeto Producto a través del constructor, el contador se incrementa. Al ser estática, esta variable es compartida por todas las instancias de la clase Producto, permitiéndonos saber cuántos productos se han creado en total.

```
public class Producto {  
    private static int contadorProductos = 0;  
    public Producto() {  
        contadorProductos++;  
    }  
    public static int getContadorProductos() {  
        return contadorProductos;  
    }  
}
```


Materiales y Recursos Adicionales



Documentación Java

Encapsulamiento y modificadores de acceso.



docs.oracle.com



Controlling Access to Members o..

This beginner Java tutorial describes fundamentals of programming in the Java programming language



Conceptos OOP

Encapsulación y colaboración.



Oracle Help Center



Java Developer's Guide

Java has emerged as the object-oriented programming language of choice. Some of the important concep...



Videos YouTube

Tutoriales sobre getters, setters y composición.



Resumen de Conceptos Clave

Encapsulamiento

Protección de datos internos

Getters y Setters

Control de acceso a atributos

Colaboración

Interacción entre objetos

Variables Estáticas

Datos compartidos por clase

Ejercicios prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, ha observado que ciertos datos del sistema se están modificando de manera impredecible. A veces, por error, algún método deja un producto con precio negativo o un cliente con email vacío. Matías y Sabrina entienden que no pueden dejar que cualquier parte del código cambie atributos sin control.

Además, ahora quieren que un **Pedido** (una nueva clase) contenga varios objetos Producto. Este escenario requiere que las clases colaboren entre sí, ya que el Pedido deberá conocer los productos que incluye.

Por último, el equipo quiere llevar la cuenta de cuántos productos se han creado en total. Esto sugiere la necesidad de una variable a nivel de clase, compartida por todas las instancias.

Para poder llegar con los tiempos de entrega dispuestos por Silvia, será necesario realizar las siguientes acciones:

Ejercicio práctico

Optativo | No entregable

1- Encapsulamiento

1) Encapsulación:

- Convertí los atributos de Cliente a private.
- Creá getters y setters para nombre y email.
- Asegurá que email contenga un @ antes de asignarlo.

2) Colaboración entre Clases:

- Creá una clase Carrito que contenga una `ArrayList<Producto>`.
- Agregale métodos para sumar productos y calcular el total.

Si te animás: demostrá el uso creando un Carrito, agregándole productos y mostrando el total.

Ejercicio práctico

Optativo | No entregable

2- Variables static

1) Variables estáticas:

- En Producto, agregá una variable estática que lleve la cuenta de cuántos productos se crean.
- Mostrá ese valor luego de instanciar varios productos.

2) Control de acceso con setters:

- Añadí una validación en el setter de cantidadEnStock para que no se permita asignar valores negativos.
- Probá asignar -10 y verificá que el valor no cambie.